D2D-Link

A chiplet link that carries AXI traffic between two dies, and never loses a transaction

Project report covering what was built, how it works and how it was verified. Written so a reader new to die-to-die links can follow it from the first page to the last.

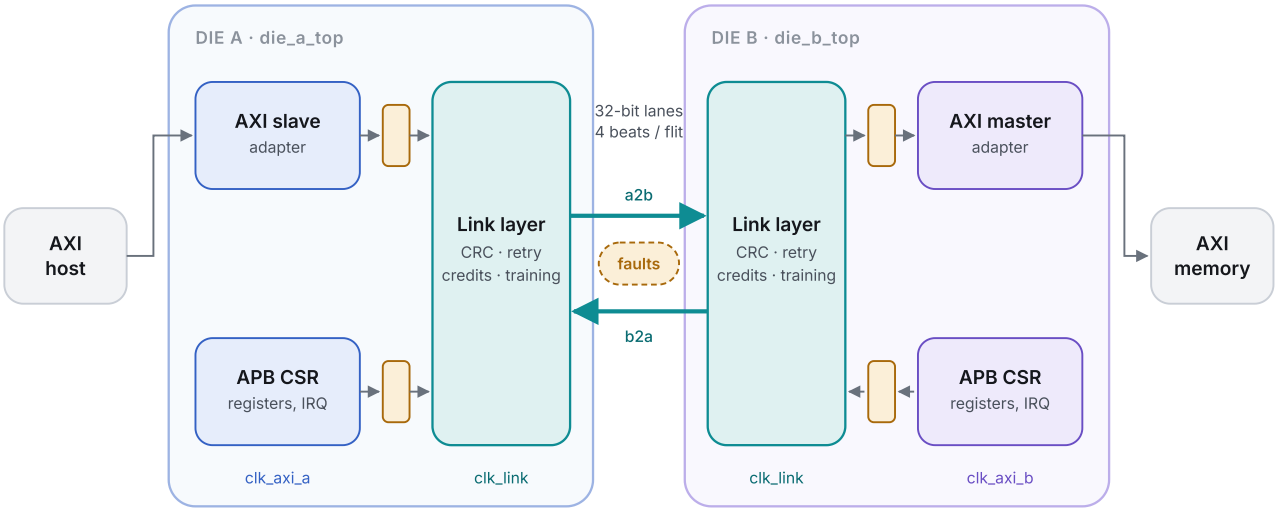
Design
UCIe-inspired die-to-die link controller

Status
Complete and verified, Nov 2026

Deliverable
RTL, testbench, proofs, reports

d2d_system_top Two dies joined by fault-injecting lanes, as simulated

44 / 44 regression jobs pass



44 / 44
regression jobs pass: 14 tests, 2 seeds, 2 configurations

100%
functional coverage, 233 of 233 bins hit

0
transactions lost, duplicated or corrupted under 3,604 injected faults

4 + 1
unbounded formal proofs plus a bounded FSM check, all PASS

01 Executive summary

Making two separate dies talk as if they were one chip

Processors are increasingly built from **chiplets**: several small dies in one package instead of one large die. The dies still have to exchange bus traffic. D2D-Link is a die-to-die link controller in synthesizable SystemVerilog. It takes AXI4 traffic on Die A, packs it into 128-bit CRC-protected packets called **flits**, sends them over a 32-bit lane and rebuilds the same transactions on Die B. When a flit is corrupted or dropped, the link recovers on its own.

The problems a die-to-die link must solve

- There are **few wires between dies**, so a wide bus must be packed into small packets and sent in beats.
- **Bits flip and packets are lost.** The link must detect this and resend, in order, without duplicates.
- The receiver has a **finite buffer**. The sender must never overflow it, even while resending.
- Each side runs on **its own clock**. Data must cross unrelated clocks without being corrupted.
- The link must **start up**, notice a bad channel, retrain, and give up cleanly if it cannot recover.

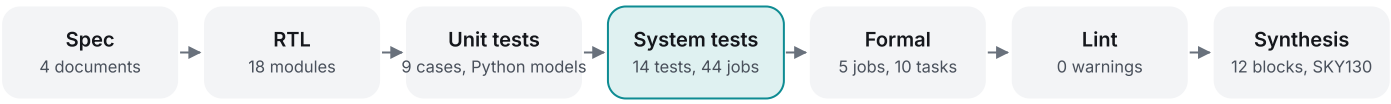
What was built

- A **specification**, microarchitecture, CDC report and verification plan, written before the RTL.
- **Synthesizable RTL**: AXI adapters, a link layer (flit TX/RX, replay buffer, credits, training FSM), async FIFOs and APB registers.
- A **pyuvm testbench** (Python UVM) with 14 constrained-random tests and fault injection on the lanes.
- **Formal proofs** in SymbiYosys of the FIFO, credits, replay buffer, a reduced retry link and the FSM.
- **Lint, synthesis** to SKY130 and a one-command Docker flow, using only open-source tools.

Headline results

44 / 44	5,108	3,604	~48 MHz
regression jobs pass in a 10-minute run on 4 parallel jobs	AXI transactions checked end to end against a reference memory	faulted flits injected (bit flips, bursts, drops), all recovered	estimated link clock in SKY130, before any timing optimisation*

How the work was done



Every number in this report is copied from a log file in the repository (docs/results.md). Re-generate everything with: make regress formal syn report

Why it matters

Die-to-die links are central to UCIe and to every chiplet product. This project takes one such link through the whole front-end flow (specification, RTL, clock crossings, UVM-style verification with fault injection, formal proofs, lint and synthesis) using only open-source tools. The interesting work is in the corner cases: a flit corrupted while the link is retraining, a lost NAK, a reset in the middle of a burst, three clocks at ratios nobody chose on purpose. The tests target exactly those cases.

Project status in one paragraph

Complete against its plan: phases 0 to 6 are committed and the working tree is clean. One real RTL bug was found by the tests and fixed. Three items are still open: the SystemVerilog UVM agent has been syntax-checked but never simulated (no SV-UVM simulator is available locally); the timing figure is an estimate, not sign-off; and the formal suite takes 709 s in total, over its 10-minute target, though every task is within its 5-minute cap.

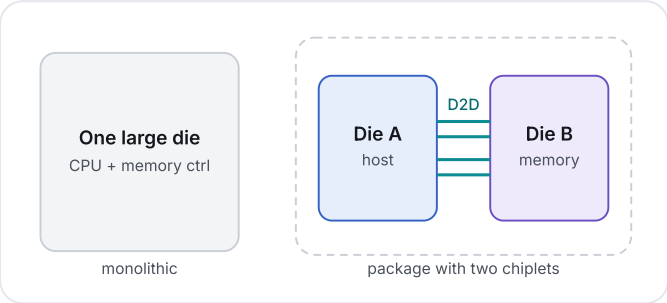
* Yosys + ABC estimate on SKY130 HD standard cells, with no wires, clock tree or placement. Treat it as an order of magnitude. D2D-Link is UCIe-*inspired*: it borrows the ideas (flits, CRC, retry, credits, training) but is not UCIe-compliant.

02 Background and specification

What a die-to-die link is, and what this one supports

A large chip can be split into smaller dies that are cheaper to make and can be reused. The price is that signals which used to be on-chip wires now have to cross between dies through a small number of package wires. A die-to-die link controller hides that boundary.

Monolithic die versus chiplets



Three layers, bottom to top

Protocol layer — UCIe: protocol

The AXI adapters turn the bus channels (AW, W, B, AR, R) into flit bodies and back again.

Link layer — UCIe: D2D adapter

Adds sequence numbers, CRC, Go-Back-N retry, credits and training, and delivers flits in order with no loss.

Physical lane model — UCIe: PHY

An ideal 32-bit parallel lane, 4 beats per flit. The testbench XORs bit flips into it and drops beats. There is no real PHY.

Feature list docs/spec.md §1.1

ID	Feature
F1	AXI4 slave on Die A, AXI4 master on Die B: 64-bit data, 32-bit address, 4-bit ID, all five channels
F2	INCR bursts of 1 to 16 beats, AxSIZE passed through, full WSTRB
F3	Up to 4 outstanding writes and 4 outstanding reads at Die A
F4	128-bit flits with CRC-16-CCITT, sent as 4 beats on a 32-bit lane
F5	Two virtual channels, request and response, each with its own credits
F6	Go-Back-N retry: 8-bit sequence numbers, 16-entry replay buffer, NAK and replay timeout
F7	Training FSM: RESET, TRAIN, ACTIVE, RETRAIN, FAILED, with no data loss across a retrain
F8	Three asynchronous clocks. Every crossing is a gray-code async FIFO
F9	APB3 register block per die: control, status, error counters, thresholds, interrupts
F10	Fault-injection ports on the lanes, in the system top only

Clock and reset domains

Domain	Clock	Contains
AXI-A	`clk_axi_a`	Die A AXI slave adapter and registers
LINK	`clk_link`	Both link layers and the lane bus
AXI-B	`clk_axi_b`	Die B AXI master adapter and registers

All three clocks are mutually asynchronous. Each reset is synchronized in its own domain: asynchronous assert, synchronous deassert through 2 flops.

Parameters default configuration and the small configuration used for formal and half the regression

Parameter	Default	Small	Meaning	Parameter	Default	Small	Meaning
`REPLAY_DEPTH`	16	4	flits kept for resend	`REPLAY_TIMEOUT`	256	128	link cycles without ACK progress
`CREDITS_REQ / RSP`	16	4	receive buffer slots per VC	`TRAIN_GOOD`	8	4	good TRAIN flits to lock
`TX_FIFO_DEPTH`	8	4	AXI to link FIFO depth	`TRAIN_TIMEOUT`	1024	512	link cycles per training attempt
`DATA_W / FLIT_W`	64 / 128	fixed	set by the flit layout	`STABLE_CYCLES`	2048	1024	ACTIVE time that clears attempts

03 System architecture

Two dies, three clocks, and every crossing goes through a FIFO

Each die has an AXI adapter and a register block in its own AXI clock domain, and a link layer in the shared link clock domain. Four async FIFOs per die are the only paths between domains. In the verification top, a fault injector sits on each lane.

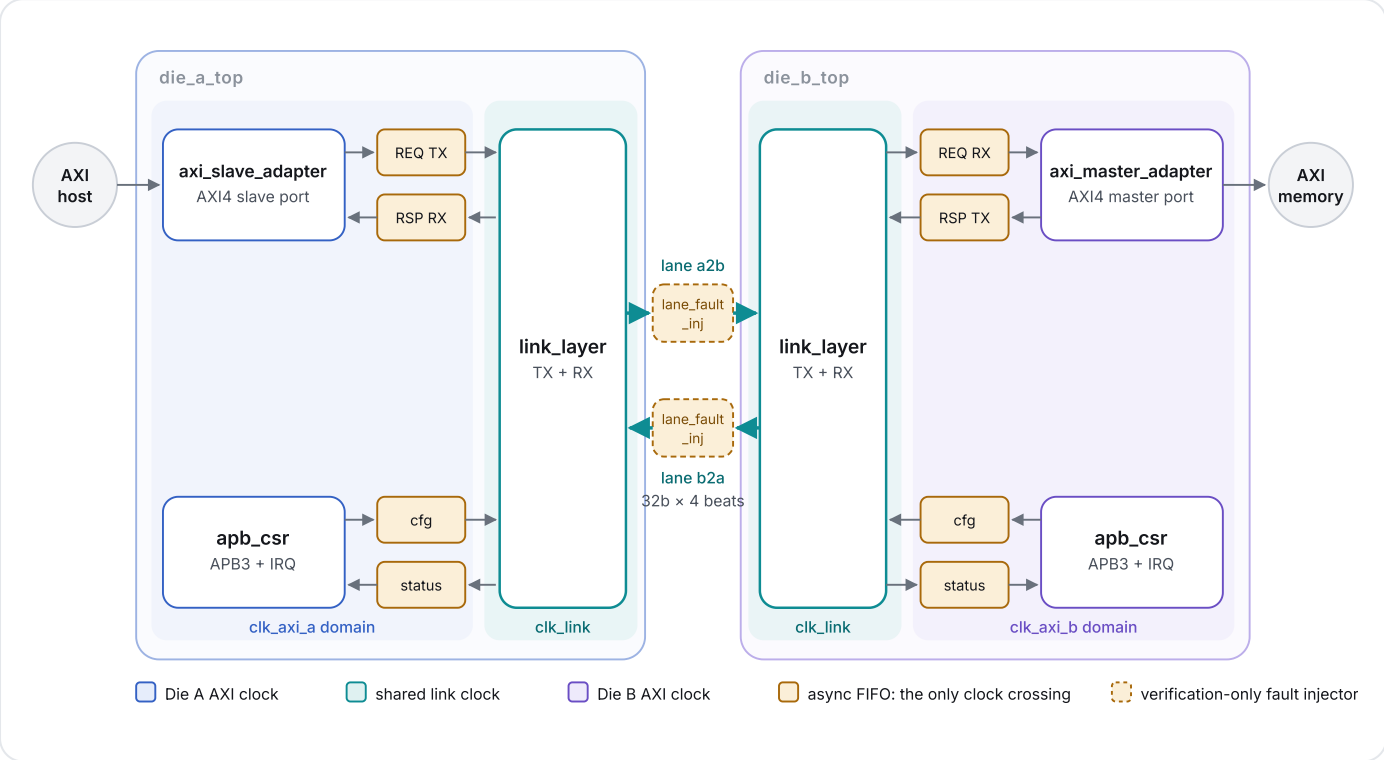


Figure 1. System block diagram (docs/microarch.md §1). Requests flow left to right on the a2b lane, responses come back on b2a. The cfg and status FIFOs carry register writes into the link domain and counter snapshots back out.

Block summary

18 modules, all synthesizable, no `initial` blocks

Module	Domain	What it does
`sync_2ff`	any	2-flop synchronizer, used only for gray pointers
`reset_sync`	any	async-assert, sync-deassert reset
`async_fifo`	2 clocks	gray-pointer FIFO, also reports the read count to the writer
`crc16`	–	parallel CRC-16-CCITT over 112 bits
`credit_counter`	link	consume 1 / return N, bounds asserted
`replay_buffer`	link	circular store of sent flits, three pointers
`flit_tx`	link	picks the next flit, fills header, CRC, serializes
`flit_rx`	link	deserializes, checks CRC and SEQ, requests NAKs

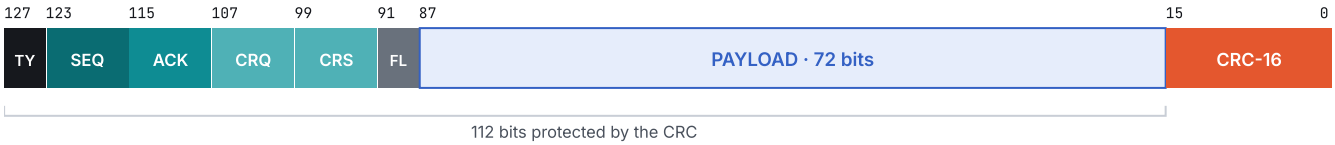
Module	Domain	What it does
`link_training_fsm`	link	training, retrain and fail state machine, error window
`link_layer`	link	integrates the above, VC arbiter, credit accumulators, counters
`axi_slave_adapter`	axi_a	AXI4 slave to REQ flit bodies, RSP bodies to B and R
`axi_master_adapter`	axi_b	REQ bodies to AXI4 master, B and R to RSP bodies
`apb_csr`	axi	registers, snapshot shadowing, interrupts
`csr_link_sync`	link	link side of the CSR crossing: config in, status snapshots out
`lane_fault_inj`	link	XOR and drop injection plus one register stage
`die_a_top / die_b_top`	all	reset syncs, CDC FIFOs, adapter, CSR, link layer
`d2d_system_top`	all	two dies and two fault injectors

04 The flit

Everything crosses the link as a 128-bit flit

A flit (flow-control unit) is a fixed-size packet. A fixed size keeps the hardware simple: the receiver always knows where a flit ends, and the CRC always covers the same bits. Every flit has a 40-bit header, a 72-bit payload and a 16-bit CRC.

Bit layout



Header fields

Field	Bits	Meaning
TYPE	127:124	which kind of flit (table right)
SEQ	123:116	sequence number, used by retry
ACK_SEQ	115:108	next SEQ expected from the other side
CRED_REQ_CUM	107:100	running total of freed request slots
CRED_RSP_CUM	99:92	same for the response channel
FLAGS	91:88	LAST beat of a burst, or LOCKED in TRAIN
PAYLOAD	87:16	72 bits, layout depends on TYPE
CRC	15:0	CRC-16/CCITT-FALSE over bits 127:16

ACK and credit fields are **running totals** (mod 256) carried on every flit, so losing one flit never loses an ACK or a credit. The next good flit brings the up-to-date value.

Nine flit types

Code	Type	Retried	VC	Carries
1	**REQ_HDR**	yes	REQ	AW or AR
2	**REQ_DATA**	yes	REQ	one W beat
3	**RSP_HDR**	yes	RSP	B response
4	**RSP_DATA**	yes	RSP	one R beat
0	**IDLE**	no	–	filler, still carries ACK and credits
5	**ACK**	no	–	sent when the ACK value changed
6	**NAK**	no	–	"resend from ACK_SEQ"
7	**CREDIT**	no	–	sent when a credit count changed
8	**TRAIN**	no	–	fixed pattern plus LOCKED bit

Codes 9 to 15 are reserved and treated as a corrupted flit.

What the 72-bit payload holds

Flit type	Payload fields, bit 87 down to 16	Used for
REQ_HDR	`is_write · id[4] · addr[32] · len[4] · size[3] · burst[2]`	one AXI AW or AR
REQ_DATA	`wdata[64] · wstrb[8] (FLAGS.LAST = wlast)`	one AXI W beat
RSP_HDR	`bid[4] · bresp[2]`	one AXI B response
RSP_DATA	`rdata[64] · rid[4] · rresp[2] (FLAGS.LAST = rlast)`	one AXI R beat
TRAIN	`72'hD2D2_A5A5_5A5A_C3C3_3C`	link training

Four beats on a 32-bit lane



One beat per `clk_link` cycle, most significant beat first. `sof` marks beat 0 so the receiver can realign. A flit cut short is counted as an error and triggers a NAK. The transmitter always sends something: IDLE, ACK or CREDIT flits fill gaps and keep ACKs and credits flowing.

CRC-16: catching corrupted flits

The transmitter runs the 112 header and payload bits through CRC-16/CCITT-FALSE (polynomial 0x1021, initial value 0xFFFF, no reflection, no final XOR) and appends the result. The receiver recomputes it. If it does not match, the flit is dropped and a NAK is sent.

CRC-16 is guaranteed to catch any 1, 2 or 3 flipped bits and any burst of up to 16 bits, so the tests inject only those patterns. In hardware it is one combinational XOR tree (`rtl/common/crc16.sv`), checked against a Python model on 1,004 random and corner-case vectors.

05 Link layer internals

Inside the link layer: a transmitter, a receiver and a memory of what was sent

The link layer is where reliability lives. The transmitter keeps a copy of every data flit until the other die acknowledges it. The receiver checks every flit and tells the transmitter what arrived, what was lost and how much buffer space has been freed.

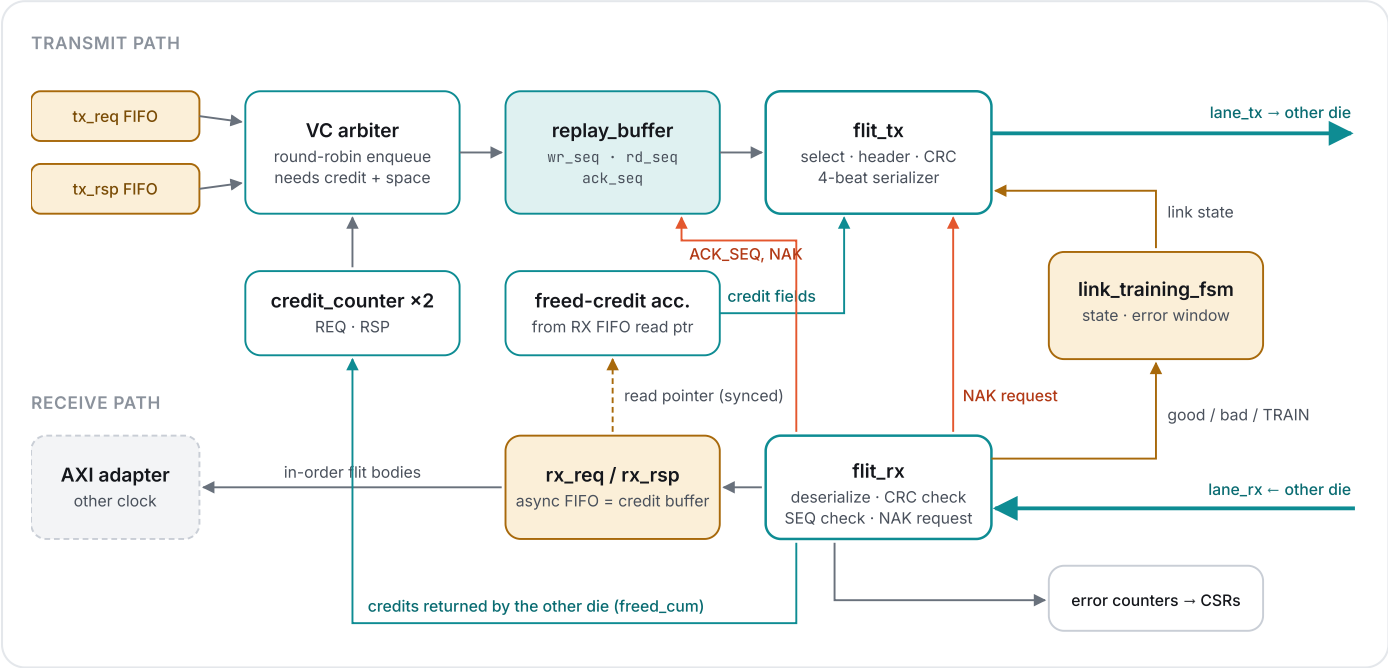


Figure 2. Link layer internals (docs/microarch.md §2). Teal: data and credits. Orange: retry control. Amber: training and CDC.

What the transmitter sends, in priority order

a. **TRAIN** while the link is training or retraining.

b. **NAK** if the receiver has requested one.

c. **The replay buffer entry at `rd_seq`**, if there is one not yet sent. That covers new data and resends.

d. **ACK** if the acknowledged value has changed.

e. **CREDIT** if a freed-credit count has changed.

f. **IDLE** otherwise. The lane is never silent while training or active.

Enqueue is separate: each cycle at most one flit body moves from a TX FIFO into the replay buffer. That needs buffer space, a credit for its channel, and a link that is not in RESET or FAILED. Enqueue assigns SEQ = `wr_seq` and spends the credit.

What the receiver does with each flit

Flit received	Action
Bad: CRC, framing, reserved type, bad TRAIN pattern	count a CRC error; in ACTIVE send one NAK for `exp_seq`
SEQ = expected	push into the RX FIFO, `exp_seq++`
SEQ ahead (a gap: a flit was lost)	request a NAK
SEQ behind (a duplicate after replay)	drop silently

Pipeline latency link cycles

Stage	Cycles
Enqueue from TX FIFO to replay buffer	1
Select, header and CRC, serialize	4 beats
Lane fault-injection register	1
Deserialize and CRC check	4 + 1
Push into RX async FIFO (+2–3 sync)	1

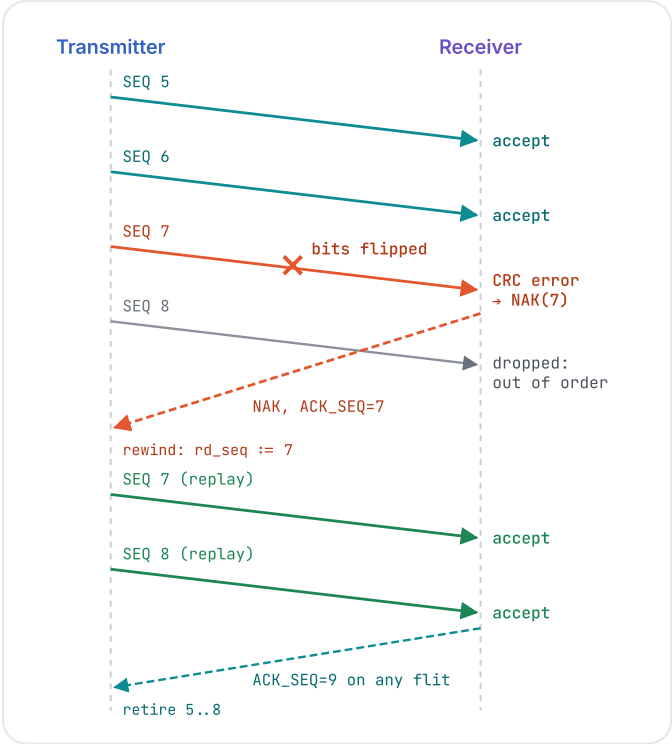
An ACK round trip takes about 24 link cycles, well under the replay timeout of 128 or 256.

06 Reliability: retry and credits

Go-Back-N retry and credits: nothing lost, nothing overflowed

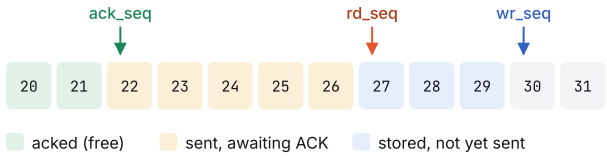
Two mechanisms make the link reliable. Retry guarantees every data flit arrives exactly once and in order. Credits guarantee the sender never overruns the receiver's buffer. Both are formally proven.

Go-Back-N retry, in action



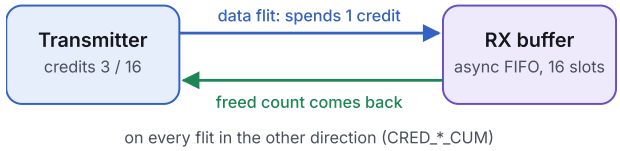
Every data flit gets a sequence number and a copy stays in the replay buffer until it is acknowledged. A bad or out-of-order flit makes the receiver send a NAK, and the transmitter goes back and resends everything from that point. If a flit simply disappears and nothing follows it, a replay timeout triggers the same rewind.

The replay buffer: three pointers



The rule is `ack ≤ rd ≤ wr` and `wr - ack ≤ 16`. There are 256 sequence numbers and the window is 16, less than half, so the receiver can always tell a duplicate (drop it) from a gap (NAK it). This invariant is formally proven.

Credits: never overflow the receiver



A credit is one free slot in the receiver's buffer, and the transmitter may only enqueue while it holds one. Request and response channels have **separate credits**, so a jammed request path can never block the responses that would clear it. That removes the classic request/response deadlock.

Three design decisions worth remembering

Cumulative, not incremental

ACK and credit fields carry running totals (mod 256). If the flit carrying them is lost, the next good flit has the up-to-date total, so a credit or ACK can never leak away.

Credits are spent once

A credit is consumed when a flit enters the replay buffer, not each time it is sent. Replays never spend credits again, so an in-order flit always finds a free slot (asserted in RTL).

Headers filled at send time

The replay buffer stores only the 80-bit body. SEQ, ACK, credits and CRC are added as the flit goes out, so a replayed flit carries fresh ACK and credit values.

Every way a flit can go wrong, and how the link recovers

Event	Detected by	Recovery	Seen in final regression
Bit flips or a corrupted beat	CRC mismatch at the receiver	NAK, then Go-Back-N replay	1,361 bad flits
Dropped data flit	SEQ gap at the next data flit	NAK, then replay	350 NAKs
Last flit, NAK or ACKs lost	replay timer expires	replay from oldest unacknowledged	62 timeouts
Error burst above threshold	error-window counter	RETRAIN, then replay	26 retrains
Retrain keeps failing	attempt counter	FAILED + interrupt; software toggles ENABLE	tested
Duplicate after a replay	SEQ behind expected	dropped	353 replays total

07 Link training and clock-domain crossing

Bringing the link up safely, and moving data between unrelated clocks

Before any data flows, both dies must agree that the link works. Each side sends TRAIN flits; after 8 good ones in a row it is **locked** and says so in its own TRAIN flits. When both sides are locked the link goes ACTIVE. Too many errors send it back to RETRAIN, and too many failed attempts end in FAILED.

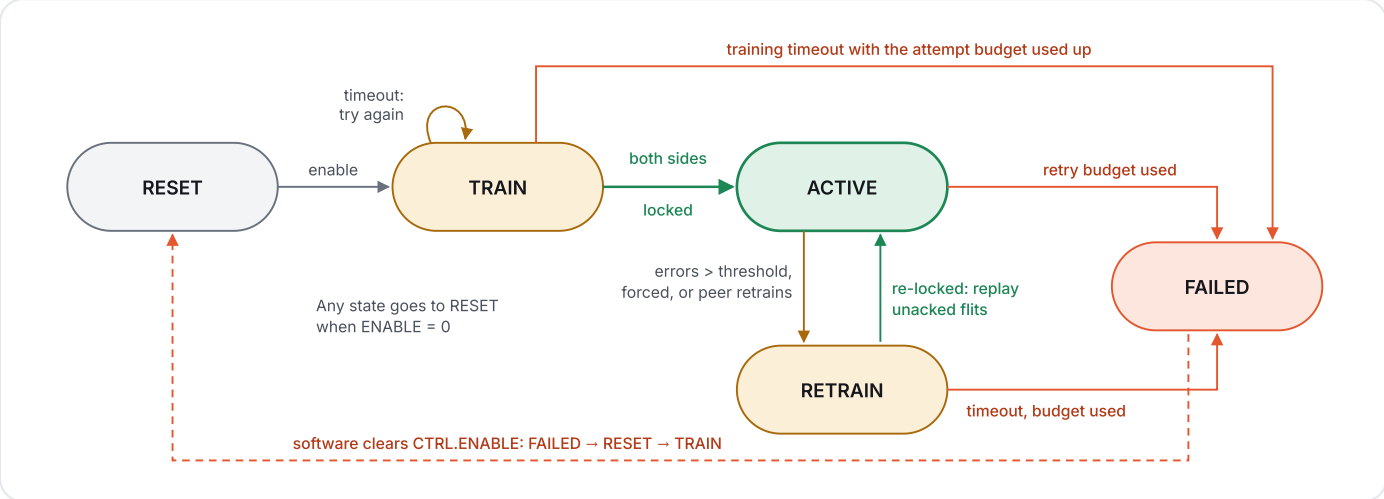


Figure 3. Link training state machine. All 5 states and 11 transition arcs are covered by the regression; ACTIVE-only-with-lock and FAILED-only-after-budget are checked by bounded model checking.

No data is lost across a retrain

Retraining never touches the replay buffer, sequence numbers, credits or receive FIFOs. On return to ACTIVE the transmitter replays everything unacknowledged and the receiver drops duplicates. A TRAIN flit with LOCKED=1 arriving while ACTIVE is ignored: without that rule the two dies could keep knocking each other back into training forever.

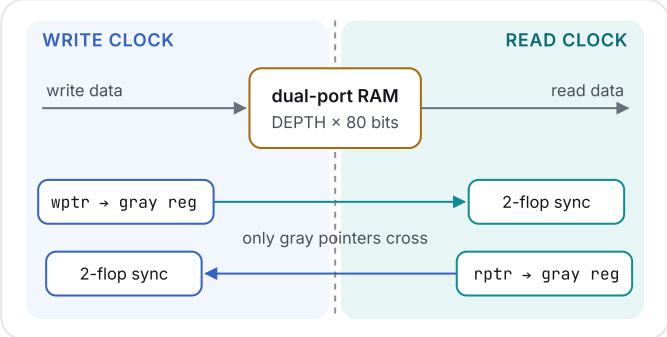
Training knobs

Knob (default)	Meaning
`ERR_THRESH = 8`	retrain if more than 8 bad flits in a window
`ERR_WINDOW = 256`	window length, in received flits
`MAX_RETRAIN = 4`	attempts before FAILED
`TRAIN_GOOD = 8`	good TRAIN flits needed to lock
`TRAIN_TIMEOUT = 1024`	link cycles per training attempt

The 8 crossings docs/cdc.md §2

Die	FIFO	Direction	Carries
A	`req_tx`	axi_a → link	AW, AR, W bodies
A	`rsp_rx`	link → axi_a	B, R bodies; RSP credit buffer
B	`req_rx`	link → axi_b	requests; REQ credit buffer
B	`rsp_tx`	axi_b → link	B, R bodies
A, B	`cfg`	axi → link	config messages (30 bits)
A, B	`sts`	link → axi	state + 5 counters (83 bits)

How a safe clock crossing works



Sampling a changing multi-bit value from another clock can capture a half-updated number. Gray code changes exactly one bit per step, so a sampled pointer is always the old or the new value, never a random third. The FIFO is formally proven with two unconstrained, unrelated clocks.

Why gray code

Step	Binary	Gray	Bits that change
3 → 4	`011 → 100`	`010 → 110`	binary 3, gray 1
7 → 0	`111 → 000`	`100 → 000`	binary 3, gray 1

Counters never cross bit by bit. The link domain pushes a complete status snapshot through a FIFO, so a register read is coherent and only a few clocks stale. **Credit return reuses the receive FIFO's** already-synchronized read pointer, so it needs no extra crossing. It lags by 2–3 cycles, so credits come back late, never early.

08 A transaction's journey

Following one AXI write burst from the host to memory and back

This is the whole design working together. A write burst is broken into flits on Die A, carried across the lane with sequence numbers and CRC, rebuilt into AXI on Die B, and its response travels back on the other virtual channel. The ACK for the request rides on the response flits.

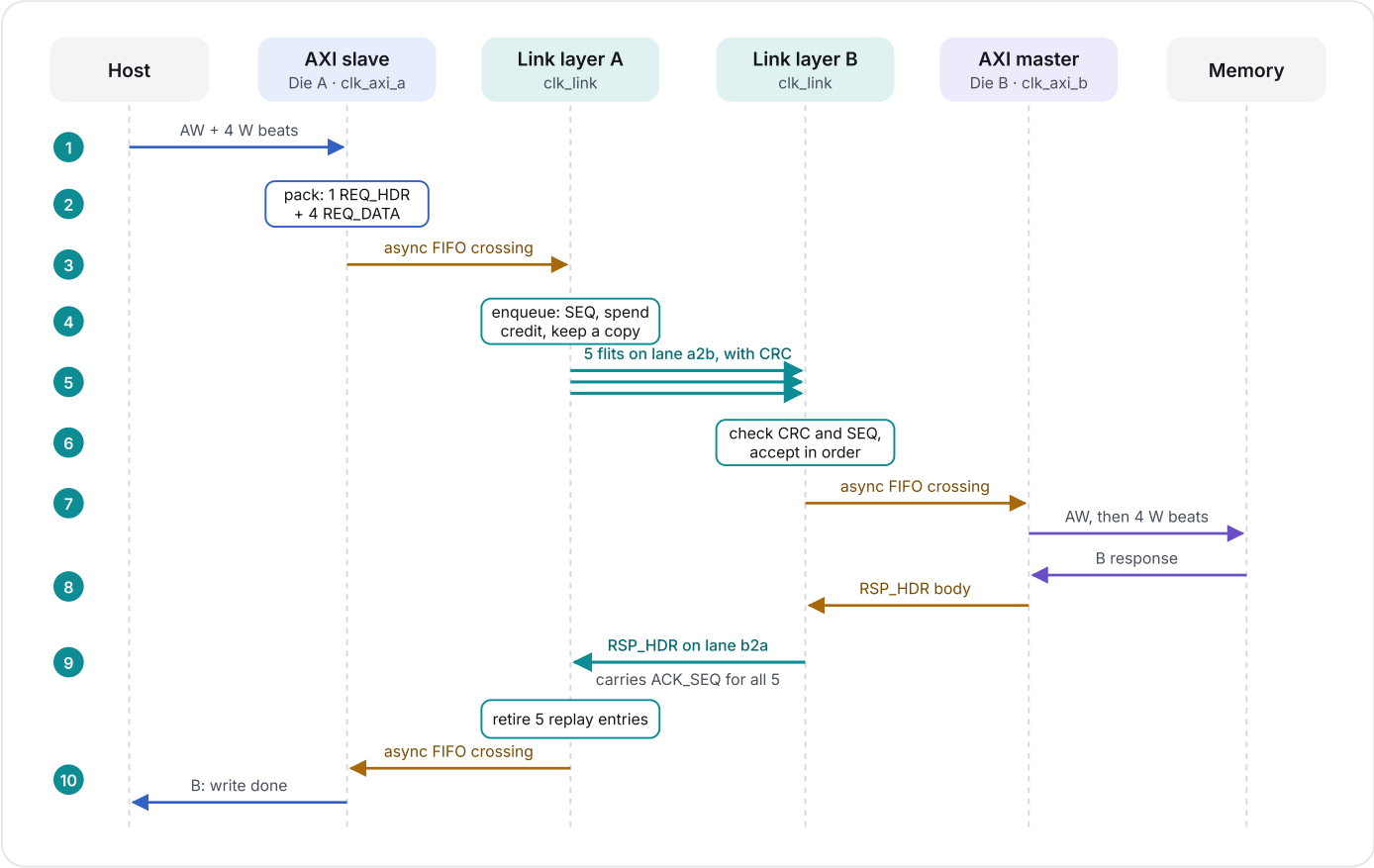


Figure 4. Sequence of one 4-beat write. Reads work the same way, with the R beats returning as RSP_DATA flits. If a flit in step 5 is corrupted, Die B sends a NAK and Die A replays from that flit; the host only sees a slightly later B.

Why one transaction at a time at Die B?

Die B executes requests strictly in arrival order, one at a time. That is legal AXI, keeps the ordering argument trivial and lets the scoreboard predict every response exactly. The cost is throughput. It is documented as a limitation; per-ID reordering would be the next feature.

Adapter rules

Die A accepts an AW only with fewer than 4 writes outstanding, and an AR only with fewer than 4 reads. AW and AR are never accepted in the same cycle, so the request order is the handshake order. Die B asserts BREADY and RREADY only when the response FIFO can take the flit.

Control and status registers

APB3, one block per die, no wait states, unmapped address → PSLVERR

Offset	Register	Access	What it does
0x00	**CTRL**	RW	ENABLE, FORCE_RETRAIN pulse
0x04	**STATUS**	RO	link state, LINK_UP
0x08	**CRC_ERR_CNT**	RO	bad flits received
0x0C	**NAK_CNT**	RO	NAKs sent
0x10	**REPLAY_CNT**	RO	replays started
0x14	**TIMEOUT_CNT**	RO	replay timeouts

Offset	Register	Access	What it does
0x18	**RETRAIN_CNT**	RO	ACTIVE → RETRAIN count
0x1C	**ERR_CFG**	RW	error threshold, retry budget
0x20	**ERR_WINDOW**	RW	error window in flits
0x24	**IRQ_EN**	RW	link down, retrain, failed, CRC
0x28	**IRQ_STATUS**	W1C	same bits, write 1 to clear
0x2C	**ID**	RO	constant 0xD2D1_0001

09 Verification

Proving it works: block tests, a UVM-style testbench, fault injection and formal

Each block was first tested on its own against a Python model. The integrated two-die system was then hammered by 14 constrained-random tests that inject faults on the lanes, while a scoreboard predicts every response. Formal proofs cover the invariants simulation can only sample.

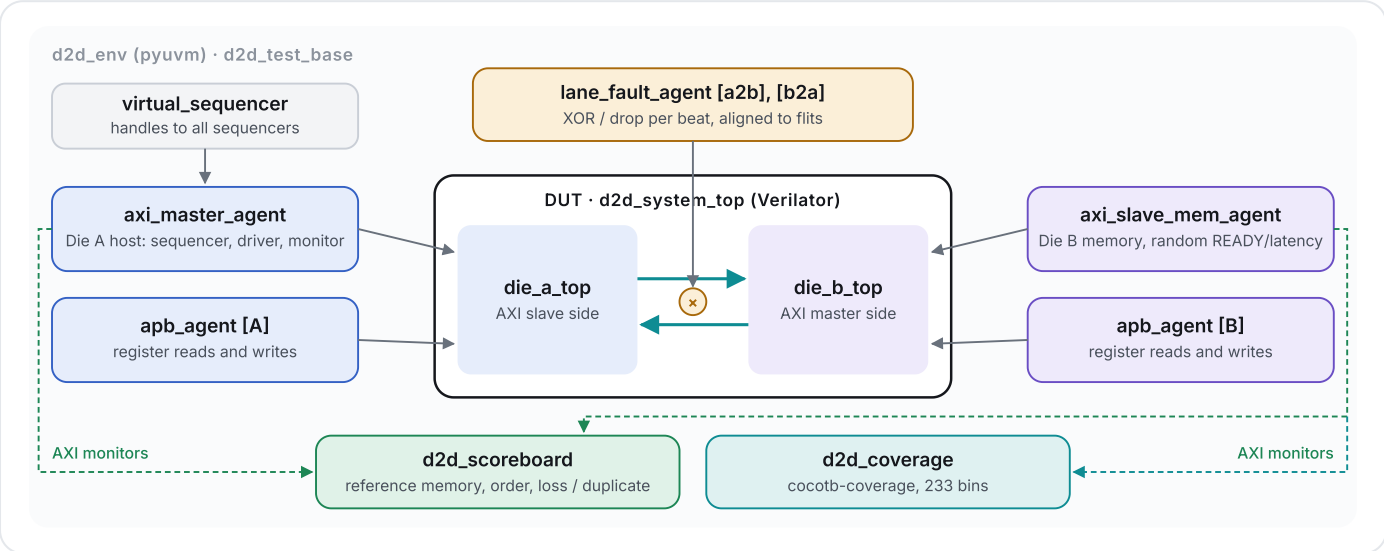


Figure 5. System testbench architecture (docs/verification_plan.md §2). Every bus-functional model drives on the falling edge and samples just before the rising edge, so combinational READY signals are seen correctly.

14 seeded system tests

Test	What it stresses
`smoke`	train, one write, one read-back
`random_rw`	200 random reads and writes, random length, ID, WSTRB
`max_burst`	16-beat bursts back to back
`outstanding_stress`	4 writes + 4 reads in flight, slow memory
`credit_starvation`	host stalls responses until credits run out
`backpressure_random`	random VALID gaps and READY at both ends
`crc_error_retry`	1% of flits get bit flips, both directions
`flit_drop_timeout`	whole flits dropped, incl. sparse tail flits
`error_storm_retrain`	heavy corruption burst forces RETRAIN
`link_fail_and_recover`	corrupt until FAILED, recover via APB
`csr_access`	reset values, RW, RO, W1C, PSLVERR
`ltssm_arcs`	directed FSM corners via registers
`clock_ratio_sweep`	several unrelated clock period triples
`reset_during_traffic`	reset mid-burst, retrain, new traffic

Red: tests that inject faults or resets. Each runs on 2 seeds; 8 also run in the small configuration, giving 44 jobs.

What the scoreboard checks

1. **Reference memory.** Writes are applied in Die A acceptance order, honouring WSTRB. Every read beat is compared against it. This is exact because execution is strictly ordered.
2. **Die B request stream.** Each request seen at Die B must equal the next Die A request. This catches loss, duplication or reordering where it happens.
3. **Responses.** ID, RESP, RLAST and beat count per burst.
4. **End of test.** Every accepted transaction completed, nothing outstanding, no extra responses, both links ACTIVE.
5. **Fault accounting.** Injected faults are logged next to the CRC, NAK, replay, timeout and retrain counters.

Unit tests 9 pytest cases, all pass

CRC against a Python model (1,004 vectors); async FIFO at depth 4 and 16 with random unrelated clocks; credit counter at every level; replay buffer against a Go-Back-N model; flit TX-to-RX loopback through accept, duplicate, gap and CRC paths; training FSM directed tests.

10 Results

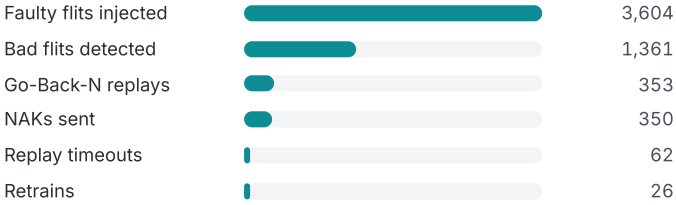
Every test passes, every coverage bin is hit, every proof holds

All numbers below are copied from log files in the repository (`logs/regress/summary.md`, `logs/formal_summary.txt`, `syn/reports/summary.txt`). Regression base seed 517299, wall clock 10.0 minutes on 4 parallel jobs.

44/44	5,108	3,604	353	26	0
jobs pass	transactions checked	faulty flits injected	Go-Back-N replays	retrains survived	lost, duplicated or corrupted

From injected fault to recovery

all 44 runs



Fewer bad flits are detected than injected, as expected. Drops remove whole flits with no CRC event (recovered by SEQ gap or timeout), and errors injected while a link is retraining or failed count only as CRC errors. One NAK can cover a run of bad flits.

Formal proofs

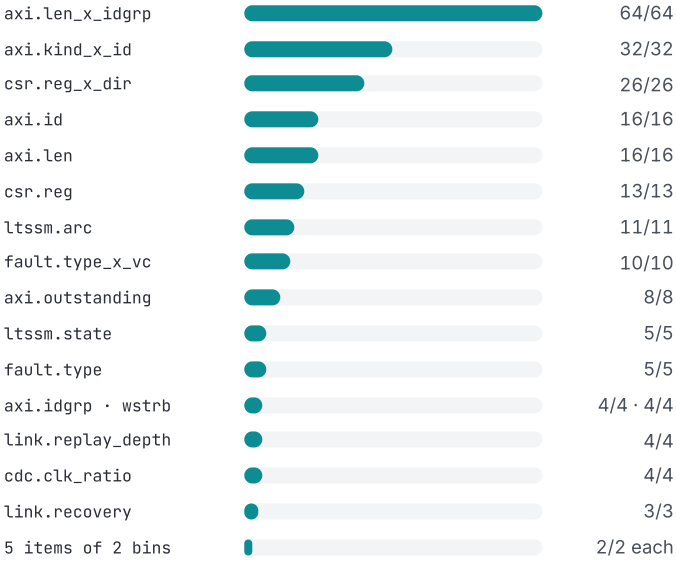
SymbiYosys, small configuration

Job	Method	What is proven	Result
fifo	ABC PDR	no overflow or underflow, 1-bit gray steps, data order, two free clocks	PASS
credit	ABC PDR	credits in [0, MAX]; conservation with late or lost returns	PASS
replay	k-induction	ack ≤ rd ≤ wr, window ≤ depth, full never accepts	PASS
retry	ABC PDR	delivered stream is exactly 0, 1, 2, … over a lossy channel	PASS
ltssm	BMC, depth 20	legal states; ACTIVE only with lock; FAILED only after budget	PASS · bounded

Each job also has a cover task, all PASS. Total 709 s, run alongside the unit tests; every task is under the 5-minute cap.

Functional coverage

233 / 233 bins, 100%



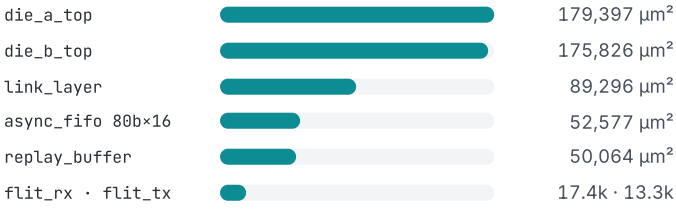
Bar length is the number of bins in each coverage item; every item is fully hit. The last row is axi.kind, csr.dir, fault.vc, link.credit_stall. An earlier run missed 4 FSM arcs and read-only-register writes; the directed `ltssm_arcs` test and new checks in `csr_access` closed them.

Lint

Tool	Result
Verilator 5.020 `--lint-only -Wall`, synthesis and `-DSIM` views	0 warnings
Verible v0.0-4298 lint	0 violations
Waivers, each justified in docs/lint_waivers.md	7

Synthesis estimates

Yosys 0.33 + ABC, SKY130 HD tt_025C_1v80, standard-cell area only, no latches inferred



Estimated clock: about 48 MHz

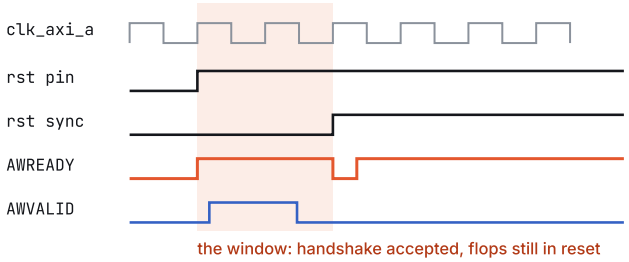
The worst combinational path is about 20.8 ns in `link_layer`: replay-buffer read mux, header assembly and the 112-bit parallel CRC (about 9.0 ns on its own). This has no wires, clock tree or setup time, and OpenSTA is not installed, so it is a rough estimate. About 2,600 of the ~4,300 flops per die are the flop-based replay and FIFO arrays.

11 Findings, limitations and next steps

What the tests found, what the design does not do, and what comes next

The bug log keeps the record honest. Real RTL defects are listed separately from testbench mistakes and from behaviour that only looked like a bug.

The real RTL bug: AXI READY high while still in reset



The pin reset deasserts 2 clocks before the internally synchronized reset. In that window the slave adapter built `AWREADY` from reset-state values, so it read 1. A host that started right after reset completed an AW handshake that the still-in-reset flops never captured: the W beats were never accepted and the bus hung. `apb_csr.pready` had the same flaw.

Caught by `reset_during_traffic`, seed 112. **Fix:** gate AWREADY, ARREADY, WREADY and PREADY with the synchronized reset. **Lesson:** every combinational handshake output must be qualified by the domain's synchronized reset, not only its registers.

Looked like a bug, was the specification

At a uniform 3% flit error rate the link went to FAILED. The retrain threshold is more than 8 bad flits per 256 (about 3.1%), so 3% regularly crosses it, retrains recur, the budget of 4 runs out and FAILED follows, exactly as specified. `crc_error_retry` now injects 1%; escalation is covered by its own tests. The thresholds encode a bit-error-rate budget, and the registers make it tunable.

Testbench bugs 5, none weakened a check

T1	BFMs sampled READY after the edge; now drive on falling edge, sample before rising
T2	loopback test released `rb_valid` a cycle early
T3	dense traffic meant NAK always beat the timeout; added a sparse phase
T4	host driver kept W beats for an AW lost to reset; added a reset epoch
T5	scoreboard counted reset-flushed transactions as missing

Known limitations docs/spec.md §9

- **Throughput:** Die B runs one transaction at a time, with no per-ID reordering.
- **Efficiency:** one flit per AXI beat: 80 useful bits of 128, 4 link cycles per beat.
- **Scope:** only INCR bursts verified; no exclusive access, QoS, cache or prot signals.
- **Reset:** both dies assumed to reset together; no single-die reset mid-traffic.
- **Recovery:** FAILED needs software to toggle ENABLE (traffic is preserved).
- **PHY:** ideal parallel lane; no deskew, scrambling or lane repair; framing trusted.
- **SV-UVM:** `uvm_sv/` is syntax-checked only, never simulated locally.

What would come next

1 · Run the SV-UVM agent

Simulate `uvm_sv/` on EDA Playground (UVM 1.2) against `die_a_top`.

2 · Raise the clock

Register the CRC check, CRC per 32-bit beat, SRAM for the replay and FIFO arrays.

3 · Raise throughput

Per-ID reordering at Die B; more than one beat per flit.

4 · Closer to a real link

PHY model, independent die resets, OpenSTA timing, code coverage.

Reproduce everything

```
make image    # build the Docker tool container
make lint     # Verilator -Wall + Verible
make unit     # block-level cocotb tests
make regress  # 44 jobs, 2 seeds, 15-min cap
make formal   # SymbiYosys proofs
make syn      # Yosys + SKY130 estimates
make report   # coverage report from logs
```

In one sentence

D2D-Link moves AXI transactions between two dies over a lossy, clock-crossing link without losing, duplicating or reordering a single one, and every claim in this report traces to a test, a proof or a log file.